

昵称：gnuemacs
园龄：5年4个月
粉丝：5
关注：1
[+加关注](#)

找找看

我的随笔
我的评论
我的参与
最新评论
我的标签

《计算机组成与设计——软硬件接口》(2)
Linux内核(2)
编译原理(2)
操作系统(1)
函数式编程(1)
体系结构(4)
信息安全(2)

2021年6月(1)
2021年3月(1)
2021年1月(4)
2020年12月(2)

SeaBIOS实现简单分析

SeaBIOS实现简单分析

SeaBIOS是一个16bit的x86 BIOS的开源实现，常用于QEMU等仿真器中使用。本文将结合SeaBIOS Execution and code flow和SeaBIOS的源码对SeaBIOS的全过程进行简单分析。需要注意，本文不是深入的分析，对于一些比较复杂和繁琐的部分直接跳过了。

从整体角度出发，SeaBIOS包含四个阶段。

- 加电自检 (Power On Self Test, POST)
- 引导 (Boot)
- 运行时 (Main runtime phase)
- 继续运行和重启 (Resume and reboot)

加电自检阶段

QEMU仿真器会将SeaBIOS加电自检阶段的第一条指令放置在F000:FFF0的位置。当QEMU仿真器启动之后，将会执行这一条指令。为什么放置在F000:FFF0位置呢？

```

+-----+ <- 0xFFFFFFFF (4GB)
|  32-bit  |
| memory mapped |
|   devices   |
|             |
|/\/\/\/\/\/\/\|
|/\/\/\/\/\/\/\|
|             |
|   Unused   |

```

2020年11月(2)

阅读排行榜

1. SeaBIOS实现简单分析(4232)
2. Linux Kernel信号处理机制源码分析(3169)
3. Shadow Stack技术概述(2737)
4. glibc动态链接实现方式(2291)
5. 几种经典的ROP手段简述(1859)

评论排行榜

1. SeaBIOS实现简单分析(2)
2. 补码的本质(1)
3. ulam——基于C语言的无类型Lambda演算解释器(1)

推荐排行榜

1. glibc动态链接实现方式(3)
2. SeaBIOS实现简单分析(3)
3. 几种经典的ROP手段简述(1)
4. Linux Kernel信号处理机制源码分析(1)
5. 补码的本质(1)

最新评论

1. Re:ulam——基于C语言的无类型Lambda演算解释器厉害啊。大部分实现使用haskell我觉得偷懒了，没想到这里有一个c的。我最近也在实现一个lambda演算解释器。不知道你的λ term用什么方式表示？直接使用AST？那样\$ \alp...

--懒人歌

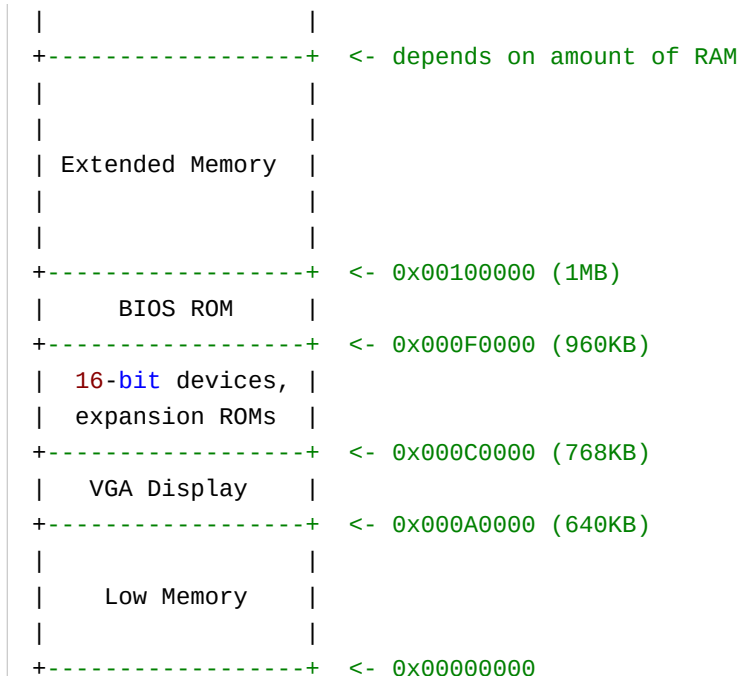
2. Re:补码的本质

写的好好

--Ryan__Liu

3. Re:SeaBIOS实现简单分析

大佬，后面两部分要提上日程了



这是从MIT 6.828 Lab1当中截取下来的一个图。PC的物理地址空间根据人们长期实践下来的约定，往往被按照上面的方式来划分。而BIOS固件，将会被放置在BIOS ROM的区域当中。这一块区域的范围是F000:0000~F000:FFFF。刚好是64KB。而为什么放在这64KB区域的顶部？因为英特尔设计8088处理器的时候，就设计成了上电启动时，将指令指针寄存器IP设置为0xFFFF0，将代码段寄存器CS设置为0xF000（指向BIOS ROM这一段）。所以，将第一条指令放在F000:FFF0位置，启动后它将立刻被执行。（实际上，这么说不是很严谨。其实CPU是从0xFFFFFFF0，也就是32bit地址线可寻址空间的最后16字节位置开始执行代码的。在刚开机的时候虽然CS为0xF000，但是它的段基址实际上是0xFFFF0000，而不是按照*16方法计算出来的0xF0000。这一点的原因在后面的“关于make_bios_writable”部分介绍）。

这个SeaBIOS里的“第一条指令”就在romlayout.S的reset_vector中。

```

reset_vector:
    jmpw $SEG_BIOS, $entry_post

```

通过这条jmp指令，程序跳转到CS:IP为SEG_BIOS:entry_post的位置。这两个是两个常量，分别为0xF000和0xE05B。而entry_post是如下定义的：

```

    ORG 0xe05b
entry_post:

```

--user20220906

4. Re:SeaBIOS实现简单分析

分析的很细致，学习了

--斯洛2019

```

    cmpl $0, %cs:HaveRunPost          // Check for resume/reboot
    jnz entry_resume
    ENTRY_INT032 _cfunc32flat_handle_post // Normal entry point

```

entry_post中，首先通过一条cmpl指令，判断是否已经经历过POST阶段。如果已经经历过该阶段，意味着当前不应该重新进行，而应该进入继续运行（Resume）。所以，如果%cs:HaveRunPost不为0，意味着已经经历过POST阶段，则进入继续运行（entry_resume），具体的过程在第四个阶段会介绍。而对于其他情况，就会进入handle_post函数。

handle_post是一个32bit的C函数，在post.c文件中。需要注意，此时机器是在16bit实模式下的。为了调用32bit的C函数，通过ENTRY_INT032，先将机器切换到保护模式，然后才能调用。

我们进一步分析ENTRY_INT032是如何实现的。ENTRY_INT032是一个宏，用于将机器切换到保护模式，然后调用一个C函数。

```

    .macro ENTRY_INT032 cfunc
    xorw %dx, %dx
    movw %dx, %ss
    movl $ BUILD_STACK_ADDR, %esp
    movl $ \cfunc, %edx
    jmp transition32
    .endm

```

可以看到，这里的cfunc是一个指向C编译器生成的函数的Label，被传递到edx寄存器中。此外，ENTRY_INT032还会设置好堆栈段寄存器SS为0，也就是将BIOS ROM程序函数调用中的堆栈保存在Low Memory区域。

transition32将使用到edx寄存器里面的值。下面是transition32的实现：

```

transition32:
    // Disable irqs (and clear direction flag)
    cli
    cld

    // Disable nmi
    movl %eax, %ecx
    movl $CMOS_RESET_CODE|NMI_DISABLE_BIT, %eax
    outb %al, $PORT_CMOS_INDEX
    inb $PORT_CMOS_DATA, %al

    // enable a20

```

```
inb $PORT_A20, %al
orb $A20_ENABLE_BIT, %al
outb %al, $PORT_A20
movl %ecx, %eax

transition32_nmi_off:
// Set segment descriptors
lidtw %cs:pmode_IDT_info
lgdtw %cs:rombios32_gdt_48

// Enable protected mode
movl %cr0, %ecx
andl $~(CR0_PG|CR0_CD|CR0_NW), %ecx
orl $CR0_PE, %ecx
movl %ecx, %cr0

// start 32bit protected mode code
ljmpl $SEG32_MODE32_CS, $(BUILD_BIOS_ADDR + 1f)

.code32
// init data segments
1: movl $SEG32_MODE32_DS, %ecx
   movw %cx, %ds
   movw %cx, %es
   movw %cx, %ss
   movw %cx, %fs
   movw %cx, %gs

   jmp 1 *%edx
```

首先，先屏蔽中断，并清空方向标志位。然后通过向一个端口写入NMI_DISABLE_BIT的方式屏蔽NMI（这里具体的不探究）。然后这一点是非常重要的——启动A20 Gate。

启动A20总线

首先将介绍A20总线。我们知道，8086/8088系列的CPU，在实模式下，按照段地址:偏移地址的方式来寻址。这种方式可以访问的最大内存地址为0xFFFF:0xFFFF，转换为物理地址0x10FFEF。而这个物理地址是21bit的，所以为了表示出这个最大的物理地址，至少需要21根地址线才能表示。

然而，8086/8088地址总线只有20根。所以在8086/8088系列的CPU上，比如如果需要寻址0x10FFEF，则会因为地址

线数目不够，被截断成0x0FFEF。再举个例子，如果要访问物理地址0x100000，则会被截断成0x00000。第21位会被省略。也就是说地址不断增长，直到0x100000的时候，会回到“0x00000”的实际物理地址。这个现象被称为“回环”现象。这种地址越界而产生回环的行为被认为是合法的，以至于当时很多程序利用到了这个特性（比如假定访问0x100000就是访问0x00000）。

然而，80286到来了。80286具有24根地址总线。对于为8086处理器设计的程序，设计者可能假定第21位会被省略。然而，在具有24根地址总线的80286机器上，则没有这个特性了。于是，如果不做出一些调整。地址总线数目的增加，可能导致向下兼容性被破坏。于是，当时的工程师们想了一个办法，设计了A20总线，用来控制第21位（如果最低位编号为0，那第21位的编号就是20）及更高位是否有效。实际上可以想象成，第21位（及更高位）都接入了一个和A20总线的与门。当A20总线为1，则高位保持原来的。当A20总线为0，则高位就始终为0。这样，当A20总线为0的时候，8086/8088的回环现象将会保持。这么一来旧程序就可以兼容了。

控制A20总线的端口被称为A20-Gate。使用in/out指令控制，即可控制A20总线是否打开。A20 Gate是0x92端口的第二个bit。先获得0x92端口的值并存放在al寄存器中，然后通过or将该寄存器的第二个bit设置为1。然后再将al的值写入0x92端口即可。这就是上面的enable a20部分的原理。

从实模式进入32位保护模式

在16bit实模式下，最多访问20根地址线。且段内偏移不能超过64KB（16位）。而32位保护模式下，则没有了最多访问20根地址线的限制，且段内偏移可以达到4GB（32位）。

此外，保护模式最大的特点是：原先的段基地址:段偏移的寻址方式，变为段选择符:段偏移的寻址方式。这里不再继续介绍保护模式，因为篇幅有限。有需要者可以自己查阅资料。

首先，前两条指令将设定中断描述符表和全局描述符表。我们重点关注全局描述符表。

```
// GDT
u64 rombios32_gdt[] VARFSEG __aligned(8) = {
    // First entry can't be used.
    0x0000000000000000LL,
    // 32 bit flat code segment (SEG32_MODE32_CS)
    GDT_GRANLIMIT(0xffffffff) | GDT_CODE | GDT_B,
    // 32 bit flat data segment (SEG32_MODE32_DS)
    GDT_GRANLIMIT(0xffffffff) | GDT_DATA | GDT_B,
    // 16 bit code segment base=0xf0000 limit=0xffff (SEG32_MODE16_CS)
    GDT_LIMIT(BUILD_BIOS_SIZE-1) | GDT_CODE | GDT_BASE(BUILD_BIOS_ADDR),
    // 16 bit data segment base=0x0 limit=0xffff (SEG32_MODE16_DS)
    GDT_LIMIT(0x0ffff) | GDT_DATA,
    // 16 bit code segment base=0xf0000 limit=0xffffffff (SEG32_MODE16BIG_CS)
    GDT_GRANLIMIT(0xffffffff) | GDT_CODE | GDT_BASE(BUILD_BIOS_ADDR),
```

```

    // 16 bit data segment base=0 limit=0xffffffff (SEG32_MODE16BIG_DS)
    GDT_GRANLIMIT(0xffffffff) | GDT_DATA,
};

// GDT descriptor
struct descloc_s rombios32_gdt_48 VARFSEG = {
    .length = sizeof(rombios32_gdt) - 1,
    .addr = (u32)rombios32_gdt,
};

```

先看（从第0项开始的）第1项

```
GDT_GRANLIMIT(0xffffffff) | GDT_CODE | GDT_B
```

这个GDT项对应的32位段基地址是0x00000000。而长度限制limit为0xFFFFFFFF。并且在32bit保护模式下偏移量也是32bit的。这意味着这个GDT项可以映射到整个物理地址空间（所以叫“Flat” code segment）。

然后Enter protected mode那里则是进入保护模式的经典方法。控制寄存器CR0的最低位（PE位）如果为1，则表示处理器处于保护模式，否则则处于实模式。我们重点关注

```
orl $CR0_PE, %ecx
```

这一个指令将PE位置为1。然后再次写入cr0寄存器。处理器即进入保护模式。下一条指令非常奇怪：

```
ljmpl $SEG32_MODE32_CS, $(BUILD_BIOS_ADDR + 1f)
```

这里的1f要区分清楚。指的是前方第一个标签为“1”的位置，而不是代表十六进制数0x1F。下一个标签“1”就是这个指令的下一条。所以，看起来这个跳转是没有价值的。实际上，在cr0寄存器被设定好之前，下一条指令已经被放入流水线。而再放入的时候这条指令还是在实模式下的。所以这个ljmp指令是为了清空流水线，确保下一条指令在保护模式下执行。

现在，我们已经在保护模式了！在这里，程序进行了这些操作：

```

.code32
// init data segments
1:  movl $SEG32_MODE32_DS, %ecx
    movw %cx, %ds
    movw %cx, %es

```

```
movw %CX, %SS
movw %CX, %FS
movw %CX, %GS
```

这里实际上含义很明确。就是初始化ds、es、ss、fs、gs寄存器，将数据段的段选择器传递给它们即可。

然后，就交给C语言编译器编译产生的代码啦！通过一个跳转指令

```
jmp *%edx
```

就完成了跳转。

从32位保护模式回到实模式

虽然没有用到这个，但是这里顺便分析一下从32位保护模式回到实模式的方法。SeaBIOS是这样实现的：

```
transition16:
    // Reset data segment limits
    movl $SEG32_MODE16_DS, %ecx
    movw %CX, %ds
    movw %CX, %es
    movw %CX, %ss
    movw %CX, %fs
    movw %CX, %gs

    // Jump to 16bit mode
    ljmpw $SEG32_MODE16_CS, $1f

    .code16
    // Disable protected mode
1:    movl %cr0, %ecx
    andl $~CR0_PE, %ecx
    movl %ecx, %cr0

    // far jump to flush CPU queue after transition to real mode
    ljmpw $SEG_BIOS, $2f

    // restore IDT to normal real-mode defaults
2:    lidt %cs:rmode_IDT_info
```

```
// Clear segment registers
xorw %cx, %cx
movw %cx, %fs
movw %cx, %gs
movw %cx, %es
movw %cx, %ds
movw %cx, %ss // Assume stack is in segment 0

jmpl *%edx
```

这里需要注意一些地方。

恢复段描述符高速缓冲寄存器

首先要了解段描述符高速缓冲寄存器（Descriptor cache register）。我们知道，GDT是存在存储器当中的。每一次在存储器中存取数据的时候，CPU需要先寻址，而寻址需要先根据段选择子计算段基址。这个计算又需要对存储器中的GDT做一次读取。于是就多了一次存储器访问，大大影响程序执行性能。所以，Intel提供的解决方案是为每一个段寄存器配备一个段描述符高速缓冲寄存器。当段寄存器被重新赋值的时候，就根据段选择子，从存储器中读取GDT中的项，然后将段基址以及其他的段描述符信息存储在这个段寄存器对应的段描述符高速缓冲寄存器中。下一次寻址的时候，就可以直接查询这个段描述符高速缓冲寄存器。性能好了很多。

然而这个寄存器，在实模式下仍然是有效的。也就是实模式下仍然会查询该寄存器以获取段基址（其实值就是当前段寄存器的值*16）。具体表格如下（出处）

实模式 下段描述符 高速缓冲 寄存器的 内容	段寄存 器	段基址 址	段界限 (固定)	段属性(固定)									
				存在 性	特权 级	已存 取	粒 度	扩展 方向	可读 性	可写 性	可执 行	堆栈 大小	一致 特权
	CS	当前 CS*16	0000FFFF H	Y	0	Y	B	U	Y	Y	Y	-	N
	SS	当前 SS*16	0000FFFF H	Y	0	Y	B	U	Y	Y	N	W	-
	DS	当前 DS*16	0000FFFF H	Y	0	Y	B	U	Y	Y	N	-	-
	ES	当前 ES*16	0000FFFF H	Y	0	Y	B	U	Y	Y	N	-	-
	FS	当前 FS*16	0000FFFF H	Y	0	Y	B	U	Y	Y	N	-	-
	GS	当前 GS*16	0000FFFF H	Y	0	Y	B	U	Y	Y	N	-	-

这就给我们一个需要注意的地方。我们在从32位保护模式切换到实模式的时候，要先把段描述符高速缓冲寄存器的内容恢复成实模式下的状态，然后再切换回去。因为在实模式下，我们无法再像保护模式中那样设定寄存器中的值了。

如何恢复呢？对于非代码段，其实很简单。因为我们段基址全部初始化成0，其他段界限等也都一样，所以只需要在GDT中新建一个表项目，也就是SeaBIOS源码中打*的这一项，然后将各个数据段寄存器设置成它即可。

```
// GDT
u64 rombios32_gdt[] VARFSEG __aligned(8) = {
    // First entry can't be used.
    0x0000000000000000LL,
    // 32 bit flat code segment (SEG32_MODE32_CS)
    GDT_GRANLIMIT(0xffffffff) | GDT_CODE | GDT_B,
    // 32 bit flat data segment (SEG32_MODE32_DS)
    GDT_GRANLIMIT(0xffffffff) | GDT_DATA | GDT_B,
    // 16 bit code segment base=0xf0000 limit=0xffff (SEG32_MODE16_CS)
    GDT_LIMIT(BUILD_BIOS_SIZE-1) | GDT_CODE | GDT_BASE(BUILD_BIOS_ADDR),
    // 16 bit data segment base=0x0 limit=0xffff (SEG32_MODE16_DS)
    // *****
    GDT_LIMIT(0x0ffff) | GDT_DATA,
    // 16 bit code segment base=0xf0000 limit=0xffffffff (SEG32_MODE16BIG_CS)
    GDT_GRANLIMIT(0xffffffff) | GDT_CODE | GDT_BASE(BUILD_BIOS_ADDR),
```

```
// 16 bit data segment base=0 limit=0xffffffff (SEG32_MODE16BIG_DS)
GDT_GRANLIMIT(0xffffffff) | GDT_DATA,
};
```

具体实现就对应了上面的“Reset data segment limits”部分的代码。

代码段寄存器的恢复

我们知道CS寄存器不能通过mov指令修改，于是就不能通过mov指令来恢复CS寄存器对应的段描述符高速缓冲寄存器了。修改CS的唯一方法是通过JMP指令。

SeaBIOS的实现是，创建了一个SEG32_MODE16_CS表项。然后通过一个ljmp指令跳转，来恢复CS寄存器。

```
ljmpw $SEG32_MODE16_CS, $1f
```

关闭PE

在上面代码的“Disable protected mode”部分，将CRO寄存器的PE位置0即刻关闭保护模式。然后，和前面一样，通过一个ljmp刷新流水线。确保后面的指令都是在实模式中运行。

进入handle_post函数

通过ENTRY_INT032_cfunc32flat_handle_post语句，即先进入保护模式，然后完成对C函数handle_post的调用。

handle_post函数的定义如下：

```
// Entry point for Power On Self Test (POST) - the BIOS initialization
// phase. This function makes the memory at 0xc0000-0xfffff
// read/writable and then calls dopost().
void VISIBLE32FLAT
handle_post(void)
{
    if (!CONFIG_QEMU && !CONFIG_COREBOOT)
        return;

    serial_debug_preinit();
    debug_banner();

    // Check if we are running under Xen.
    xen_preinit();
```

```
// Allow writes to modify bios area (0xf0000)
make_bios_writable();

// Now that memory is read/writable - start post process.
dopost();
}
```

首先是一些基本的准备工作，比如启动串口调试等等，这些细节我们就忽略了。从make_bios_writable开始。

关于make_bios_writable

这里不放代码，仅仅简单介绍该函数的作用——允许更改RAM中的BIOS ROM区域。在介绍make_bios_writable之前，首先对Shadow RAM做一些介绍。实际上，尽管在启动的时候，是从F000:FFF0加载第一条指令的，你可能会觉得在启动的时候代码段基址是0xF0000。其实，并不是这样的。在计算机启动的时候，代码段基址实际上是0xFFFF0000（这里就不符合那个乘16的计算方式了）。笔者猜测这一点的实现方式是通过段描述符高速缓冲寄存器实现的（实模式下也是通过查询这个寄存器来获得段基址的），开机的时候代码段的对应基址项被设置成0xFFFF0000。

为什么从这里开始呢？我们知道BIOS是存储在ROM当中的。而Intel有一个习惯，将BIOS固件代码从ROM中映射到可寻址地址的末端（最后64K内）。这里的“映射”，并不是复制，而是当读取这个地址的时候，就直接读取ROM存储器当中的值。在8086时期，可寻址的地址为0x00000-0xFFFFF，所以说它的“末端”确实是从我们理解的0xF0000开始的。所以在8086时期，硬件设备将会将原本存储于ROM的BIOS映射到F000:0000-F000:FFFF。然而，到了后面有32根地址线，实际上末端应该是0xFFFF0000-0xFFFFFFFF这一部分。此时的计算机，实际上是将BIOS固件代码映射到0xFFFF0000-0xFFFFFFFF中。

所以，实际上SeaBIOS的这一行指令：

```
reset_vector:
    ljmpw $SEG_BIOS, $entry_post
```

是位于0xFFFFFFFF的物理地址位置的。但是我们注意到这是一个Long jump指令，这个指令会使CPU重新计算代码段寄存器，原本的0xFFFF0000基址，在这一个指令执行之后，就会变成符合乘16计算方式的0xF0000！

读者可能会想，这不就出问题了吗？32根地址线的PC，BIOS固件明明在最后呀！实际上，为了保持向前兼容性，机器启动的时候会自动将ROM的BIOS复制到RAM的BIOS ROM区域当中。所以，通过ljmpw指令跳转之后，因为已经复制了，在RAM当中也有BIOS固件代码。所以是不会有问题的。

这个复制高地址处的ROM到低地址处的过程被称为Shadow RAM技术。然而，在这个过程后，这段内存会被保护起来，无法进行写入。make_bios_writable函数就用于让这段内存可写，从而便于更改一些静态分配的全局变量值。

进入dopost

刚才已经做好了准备。然后，就可以进入dopost函数了。这个函数是POST过程的主体。dopost当中，将会调用maininit函数。下面的“maininit过程”，将作详细介绍。

maininit过程

dopost函数定义如下

```
// Setup for code relocation and then relocate.
void VISIBLE32INIT
dopost(void)
{
    code_mutable_preinit();

    // Detect ram and setup internal malloc.
    qemu_preinit();
    coreboot_preinit();
    malloc_preinit();

    // Relocate initialization code and call maininit().
    reloc_preinit(maininit, NULL);
}
```

首先，看code_mutable_preinit。

```
void
code_mutable_preinit(void)
{
    if (HaveRunPost)
        // Already run
        return;
    // Setup reset-vector entry point (controls legacy reboots).
    rtc_write(CMOS_RESET_CODE, 0);
    barrier();
    HaveRunPost = 1;
    barrier();
}
```

这一段的核⼼是将HaveRunPost设置为1。可以看出，HaveRunPost实际上相当于一个全局变量，在BIOS ROM中实际上是被初始化为0的。然后将ROM映射到RAM中的BIOS ROM区域之后，通过make_bios_writable，使得这一段RAM区域可写，然后才能更改HaveRunPost的值。

为了初始化内存，SeaBIOS实现了自己的malloc函数。通过malloc_preinit进行初始化，然后通过reloc_preinit函数将自身代码进行重定位。这些步骤有非常多的工程细节，就忽略不看了。接下来从重要的函数：maininit开始分析。

```
// Main setup code.
static void
maininit(void)
{
    // Initialize internal interfaces.
    interface_init();

    // Setup platform devices.
    platform_hardware_setup();

    // Start hardware initialization (if threads allowed during optionroms)
    if (threads_during_optionroms())
        device_hardware_setup();

    // Run vga option rom
    vgarom_setup();
    sercon_setup();
    enable_vga_console();

    // Do hardware initialization (if running synchronously)
    if (!threads_during_optionroms()) {
        device_hardware_setup();
        wait_threads();
    }

    // Run option roms
    optionrom_setup();

    // Allow user to modify overall boot order.
    interactive_bootmenu();
    wait_threads();
}
```

```
// Prepare for boot.
prepareboot();

// Write protect bios memory.
make_bios_readonly();

// Invoke int 19 to start boot process.
startBoot();
}
```

maininit函数中，有很多重要的部分。我们来看一看。

首先是interface_init函数。

```
void
interface_init(void)
{
    // Running at new code address - do code relocation fixups
    malloc_init();

    // Setup romfile items.
    qemu_cfg_init();
    coreboot_cbfs_init();
    multiboot_init();

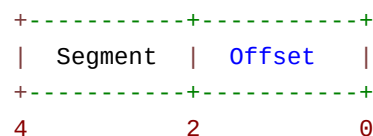
    // Setup ivt/bda/ebda
    ivt_init();
    bda_init();

    // Other interfaces
    boot_init();
    bios32_init();
    pmm_init();
    pnp_init();
    kbd_init();
    mouse_init();
}
```

这个函数用于加载内部的一些接口。下面是其步骤。

初始化中断向量表IVT

中断向量表 (Interrupt Vector Table) 是一张在实模式下使用的表。顾名思义, 这个表将中断号映射到中断过程的一个列表。中断向量表必须存储在低地址区域 (也就是从0x00000000) 开始, 大小一般是0x400字节。是一块由很多项组成的连续的内存空间。每一项, 就对应了一个中断, 如下所示 (出处) :



每一项被称为中断向量 (Interrupt Vector) 。笔者认为因为每一项都可以写成(segment, offset)的形式, 仿佛一个二维向量坐标, 所以被称为“中断向量”。可以看出每一项占据4个字节, 前两个字节是段, 后两个字节是偏移。而这一项实际上就对应了一个中断服务处理程序的入口地址 (段:偏移)。只需要修改这个表里的地址, 即可更换中断处理程序。

而每一个中断都有一个中断号码, 号码就是中断向量的索引。比如这个表里前四个字节对应的项, 中断号就是0, 然后4-8字节对应的项中断号就是1。可以很容易地看出, 中断号*4为首地址的4字节内存区域就对应了该中断号对应的中断处理程序位置。

SeaBIOS中, ivt_init就是初始化一些中断。我们看实现。

```
static void
ivt_init(void)
{
    dprintf(3, "init ivt\n");

    // Initialize all vectors to the default handler.
    int i;
    for (i=0; i<256; i++)
        SET_IVT(i, FUNC16(entry_iret_official));

    // Initialize all hw vectors to a default hw handler.
    for (i=BIOS_HWIRQ0_VECTOR; i<BIOS_HWIRQ0_VECTOR+8; i++)
        SET_IVT(i, FUNC16(entry_hwpic1));
    for (i=BIOS_HWIRQ8_VECTOR; i<BIOS_HWIRQ8_VECTOR+8; i++)
        SET_IVT(i, FUNC16(entry_hwpic2));

    // Initialize software handlers.
    SET_IVT(0x02, FUNC16(entry_02));
```

```
SET_IVT(0x05, FUNC16(entry_05));
SET_IVT(0x10, FUNC16(entry_10));
SET_IVT(0x11, FUNC16(entry_11));
SET_IVT(0x12, FUNC16(entry_12));
SET_IVT(0x13, FUNC16(entry_13_official));
SET_IVT(0x14, FUNC16(entry_14));
SET_IVT(0x15, FUNC16(entry_15_official));
SET_IVT(0x16, FUNC16(entry_16));
SET_IVT(0x17, FUNC16(entry_17));
SET_IVT(0x18, FUNC16(entry_18));
SET_IVT(0x19, FUNC16(entry_19_official));
SET_IVT(0x1a, FUNC16(entry_1a_official));
SET_IVT(0x40, FUNC16(entry_40));

// INT 60h-66h reserved for user interrupt
for (i=0x60; i<=0x66; i++)
    SET_IVT(i, SEGOFF(0, 0));

// set vector 0x79 to zero
// this is used by 'gardian angel' protection system
SET_IVT(0x79, SEGOFF(0, 0));
}
```

首先，ivt_init将所有中断初始化到一个空处理函数（相当于只有一条return语句）。将所有硬中断也都初始化到一个默认处理函数。然后是一些默认的中断处理程序。比如经典的VGA服务INT 10H，经典的磁盘服务INT 13H等等。对于每一项的事先，这里不多介绍。

初始化BIOS数据区域BDA

BDA（BIOS Data Area），是存放计算机当前一些状态的位置。在SeaBIOS中，其定义如下：

```
struct bios_data_area_s {
    // 40:00
    u16 port_com[4];
    u16 port_lpt[3];
    u16 ebda_seg;
    // 40:10
    u16 equipment_list_flags;
    u8 pad1;
```



```
u16 mem_size_kb;
u8 pad2;
u8 ps2_ctrl_flag;
u16 kbd_flag0;
u8 alt_keypad;
u16 kbd_buf_head;
u16 kbd_buf_tail;
// 40:1e
u8 kbd_buf[32];
u8 floppy_recalibration_status;
u8 floppy_motor_status;
// 40:40
u8 floppy_motor_counter;
u8 floppy_last_status;
u8 floppy_return_status[7];
u8 video_mode;
u16 video_cols;
u16 video_pagesize;
u16 video_pagestart;
// 40:50
u16 cursor_pos[8];
// 40:60
u16 cursor_type;
u8 video_page;
u16 crtc_address;
u8 video_msr;
u8 video_pal;
struct segoff_s jump;
u8 other_6b;
u32 timer_counter;
// 40:70
u8 timer_rollover;
u8 break_flag;
u16 soft_reset_flag;
u8 disk_last_status;
u8 hdcount;
u8 disk_control_byte;
u8 port_disk;
u8 lpt_timeout[4];
u8 com_timeout[4];
```

```
// 40:80
u16 kbd_buf_start_offset;
u16 kbd_buf_end_offset;
u8 video_rows;
u16 char_height;
u8 video_ctl;
u8 video_switches;
u8 modeset_ctl;
u8 dcc_index;
u8 floppy_last_data_rate;
u8 disk_status_controller;
u8 disk_error_controller;
u8 disk_interrupt_flag;
u8 floppy_harddisk_info;
// 40:90
u8 floppy_media_state[4];
u8 floppy_track[2];
u8 kbd_flag1;
u8 kbd_led;
struct segoff_s user_wait_complete_flag;
u32 user_wait_timeout;
// 40:A0
u8 rtc_wait_flag;
u8 other_a1[7];
struct segoff_s video_savetable;
u8 other_ac[4];
// 40:B0
u8 other_b0[5*16];
} PACKED;
```

bda_init就是初始化该区域的函数。这里不多介绍。

BOOT阶段前最后的准备

然后是一些其他的接口，比如键盘、鼠标接口的加载。在interface_init函数的最后部分定义。这里不再介绍。在maininit函数中，剩余的部分

```
// Run vga option rom
vgarom_setup();
sercon_setup();
```

```
enable_vga_console();

// Do hardware initialization (if running synchronously)
if (!threads_during_optionroms()) {
    device_hardware_setup();
    wait_threads();
}

// Run option roms
optionrom_setup();

// Allow user to modify overall boot order.
interactive_bootmenu();
wait_threads();

// Prepare for boot.
prepareboot();

// Write protect bios memory.
make_bios_readonly();

// Invoke int 19 to start boot process.
startBoot();
```

用于加载VGA设备、初始化硬件、为用户提供更改启动顺序的界面。然后，将刚才被设置为可写的RAM中的BIOS ROM部分，重新保护起来。然后，通过一个startBoot函数，调用INT19中断，进入Boot状态。

startBoot

```
void VISIBLE32FLAT
startBoot(void)
{
    // Clear low-memory allocations (required by PMM spec).
    memset((void*)BUILD_STACK_ADDR, 0, BUILD_EBDA_MINIMUM - BUILD_STACK_ADDR);

    dprintf(3, "Jump to int19\n");
    struct bregs br;
    memset(&br, 0, sizeof(br));
    br.flags = F_IF;
```

```
    call16_int(0x19, &br);  
}
```

当前，CPU出于保护模式。但是在引导至OS Bootloader之前，需要告别保护模式，回到实模式。call16_int使用之前提到的transition16回到实模式之后，调用INT 19H中断，进入BOOT状态。

引导阶段

boot阶段的代码，是从handle_19，也就是的0x19中断处理程序开始。

```
void VISIBLE32FLAT  
handle_19(void)  
{  
    debug_enter(NULL, DEBUG_HDL_19);  
    BootSequence = 0;  
    do_boot(0);  
}
```

do_boot函数：

```
static void  
do_boot(int seq_nr)  
{  
    if (! CONFIG_BOOT)  
        panic("Boot support not compiled in.\n");  
  
    if (seq_nr >= BEVCount)  
        boot_fail();  
  
    // Boot the given BEV type.  
    struct bev_s *ie = &BEV[seq_nr];  
    switch (ie->type) {  
    case IPL_TYPE_FLOPPY:  
        printf("Booting from Floppy...\n");  
        boot_disk(0x00, CheckFloppySig);  
        break;  
    case IPL_TYPE_HARDDISK:  
        printf("Booting from Hard Disk...\n");
```

```
        boot_disk(0x80, 1);
        break;
    case IPL_TYPE_CDROM:
        boot_cdrom((void*)ie->vector);
        break;
    case IPL_TYPE_CBFS:
        boot_cbfs((void*)ie->vector);
        break;
    case IPL_TYPE_BEV:
        boot_rom(ie->vector);
        break;
    case IPL_TYPE_HALT:
        boot_fail();
        break;
}

// Boot failed: invoke the boot recovery function
struct bregs br;
memset(&br, 0, sizeof(br));
br.flags = F_IF;
call16_int(0x18, &br);
}
```

可以发现这里为从软盘、硬盘、CDROM等一系列设备中引导提供了支持。我们重点关注从硬盘引导。

```
static void
boot_disk(u8 bootdrv, int checksig)
{
    u16 bootseg = 0x07c0;

    // Read sector
    struct bregs br;
    memset(&br, 0, sizeof(br));
    br.flags = F_IF;
    br.dl = bootdrv;
    br.es = bootseg;
    br.ah = 2;
    br.al = 1;
    br.cl = 1;
```

```
call16_int(0x13, &br);

if (br.flags & F_CF) {
    printf("Boot failed: could not read the boot disk\n\n");
    return;
}

if (checksig) {
    struct mbr_s *mbr = (void*)0;
    if (GET_FARVAR(bootseg, mbr->signature) != MBR_SIGNATURE) {
        printf("Boot failed: not a bootable disk\n\n");
        return;
    }
}

tpm_add_bcv(bootdrv, MAKE_FLATPTR(bootseg, 0), 512);

/* Canonicalize bootseg:bootip */
u16 bootip = (bootseg & 0x0fff) << 4;
bootseg &= 0xf000;

call_boot_entry(SEGOFF(bootseg, bootip), bootdrv);
}
```

读取主引导扇区

首先，通过调用0x13中断读取主引导扇区。读扇区的参数定义如下：

功能描述：读扇区

入口参数：AH = 02H

AL = 扇区数

CH = 柱面

CL = 扇区

DH = 磁头

DL = 驱动器，00H~7FH：软盘；80H~0FFH：硬盘

ES:BX = 缓冲区的地址

出口参数：CF = 0——操作成功，AH = 00H，AL = 传输的扇区数，否则，AH = 状态代码

首先，需要了解（复习）磁盘的CHS模式。Cylinder-head-sector（柱面-磁头-扇区，CHS）是一个定位磁盘数据位置的方法。因为向磁盘读取数据，要先告诉磁盘读取哪里数据。

下面的两张图片选自Wikipedia。

Cylinder, head, and sector of a hard drive.

首先介绍磁头。一个磁盘可以看成多个重叠的盘片组成。磁头可以选择读取哪一个盘片上的数据。

其次是柱面。在某一个盘片上，找一个同心圆，这个圆对应的圈叫做磁道。而把所有盘片叠在一起，所有磁道构成的面叫做柱面。所以实际上柱面可以指定在当前磁头所指向的盘片上磁道的“半径”。

然后是扇区。扇区的概念非常重要。一个磁道上连续的一段称为扇区。每个磁道被等分为若干个扇区。一般来说，一个扇区包含512字节的数据。

磁头、柱面都是从0开始编号的。扇区是从1开始编号的。

从代码的Read sector部分可以看出，boot_disk将读取使用bootdrv指定的磁盘驱动器上，0磁头，0柱面，1扇区为起始位置，扇区数为1（512字节）的一段数据。然后将这段部分复制到0x7c00的内存地址当中。这个内存地址可谓是非常经典。

在checksig的部分，GET_FARVAR那一句的含义就是以bootseg为段来读取mbr中的signature。而mbr此时指向的地址偏移量为0。我们看mbr_s数据结构的定义：

```
struct mbr_s {
    u8 code[440];
    // 0x01b8
    u32 diskseg;
    // 0x01bc
    u16 null;
    // 0x01be
    struct partition_s partitions[4];
    // 0x01fe
    u16 signature;
} PACKED;
```

可以看出signature是MBR主引导扇区代码的最后两个字节。这里又是一大经典。可以看出，checksig的部分是用于校验主引导扇区代码的最后两个字节是否为MBR_SIGNATURE。而这个值恰恰就是那个经典的数字：0xAA55。

接近尾声

引导过程快要接近尾声了。我们注意到一个“规范化”（Canonicalize）操作：

```
/* Canonicalize bootseg:bootip */
u16 bootip = (bootseg & 0x0fff) << 4;
bootseg &= 0xf000;
```

这个操作实际上是为了调用OS Loader的时候，段为0x0000，而偏移为0x7C00。而不是段为0x07C0，而偏移为0x0000。

最后，通过一个call_boot_entry函数，转移到OS Loader。此时，系统处于实模式下。

```
static void
call_boot_entry(struct segoff_s bootsegip, u8 bootdrv)
{
    dprintf(1, "Booting from %04x:%04x\n", bootsegip.seg, bootsegip.offset);
    struct bregs br;
    memset(&br, 0, sizeof(br));
    br.flags = F_IF;
    br.code = bootsegip;
    // Set the magic number in ax and the boot drive in dl.
    br.dl = bootdrv;
    br.ax = 0xaa55;
    farcall16(&br);
}
```

这就是引导部分的主要内容。

剩余部分

因为最近事情比较多，最后两部分还没有写完。后面更新。

[好文要顶](#)[关注我](#)[收藏该文](#)[微信分享](#)

gnuemacs

粉丝 - 5 关注 - 1

[+加关注](#)

3

0

[升级成为会员](#)

« 上一篇：补码的本质

» 下一篇：Linux Kernel信号处理机制源码分析

posted on 2021-01-16 20:02 gnuemacs 阅读(4232) 评论(2) 收藏 举报

博客园 © 2004-2025